



Monitorea tu servidor local con telegraf, influxdb y grafana

¡Hola, soy Jorge! Un poco sobre mi



Graduado de Ingeniería electrónica.

Desarrollador Backend e Ingeniero de Datos.

Me gusta la programación, las matemáticas, el software libre, fotografiar y dibujar.

Los servidores usualmente ejecutan múltiples aplicaciones, las cuales consumen recursos: CPU, RAM, Disco, Ancho de Banda.

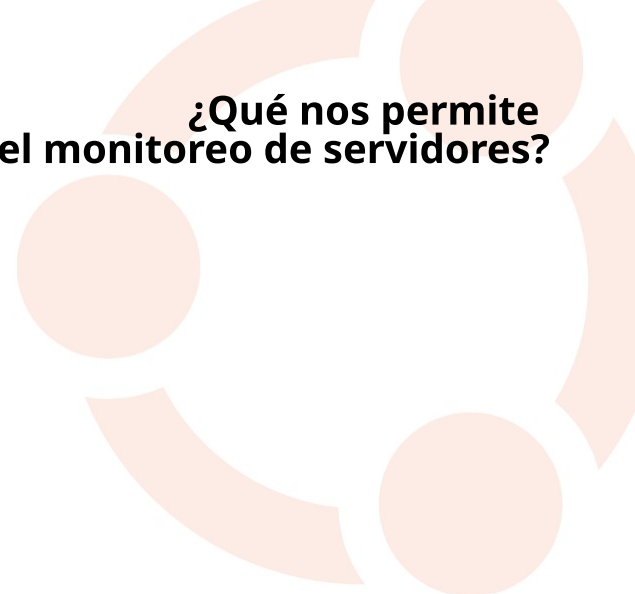
El monitoreo nos muestra si algún recurso está siendo llevado al límite.

Una pequeña introducción al monitoreo



Fuente Imagen: [Pixabay](#)

1. **Detección de problemas** en etapas tempranas.
2. **Optimización** del rendimiento de las aplicaciones.
3. Mejora los procesos de **seguridad**.
4. Mantener la **disponibilidad** de los servicios.



¿Qué nos permite
el monitoreo de servidores?

Y de las más importantes...
Prevenir

- Caídas de servicio.
- Pérdida de datos.
- Fallos en el hardware.

**¿Qué podemos
prevenir a través del monitoreo?**

- Problemas de rendimiento.
- Vulnerabilidades de seguridad.

¿Cuáles son los principales recursos que usualmente son monitoreados?

CPU
Uso de CPU
Procesos activos y en cola

RAM
Memoria RAM usada/disponible
Memoria virtual

Disco
Unidades disponibles
Espacio disponible
Número de innodos vs disponibles

RED
Ancho de banda.
Tráfico de red UDP, TCP
Interfaces de red

Temperatura
CPU y GPU
Discos
Board

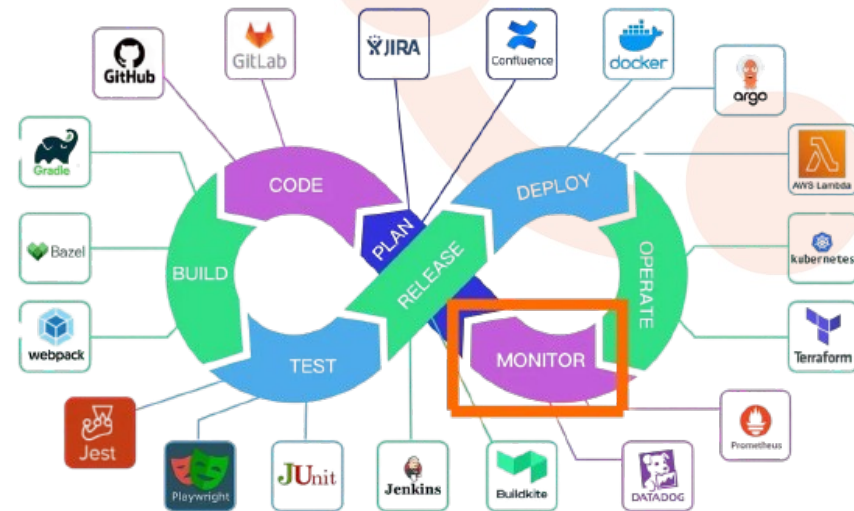
Logs del sistema
NLP a los logs
Frecuencia de los logs de advertencia

Servicios y procesos
Estado de los servicios
Cantidad y estado de los procesos.

El monitoreo proporciona **retroalimentación** sobre el estado del sistema. Esta retroalimentación facilita el proceso de **automatización**.

El monitoreo se integra con el **escalamiento** vertical de recurso y ejecución de **procesos de mitigación** para despliegues fallidos.

Monitoreo y CI/CD



Fuente imagen: Bytebytego

En aplicaciones y modelos de machine learning.

- Precisión de los modelos.
- Matriz de predicciones y Recall.
- Pérdidas.
- Tiempo de respuesta.

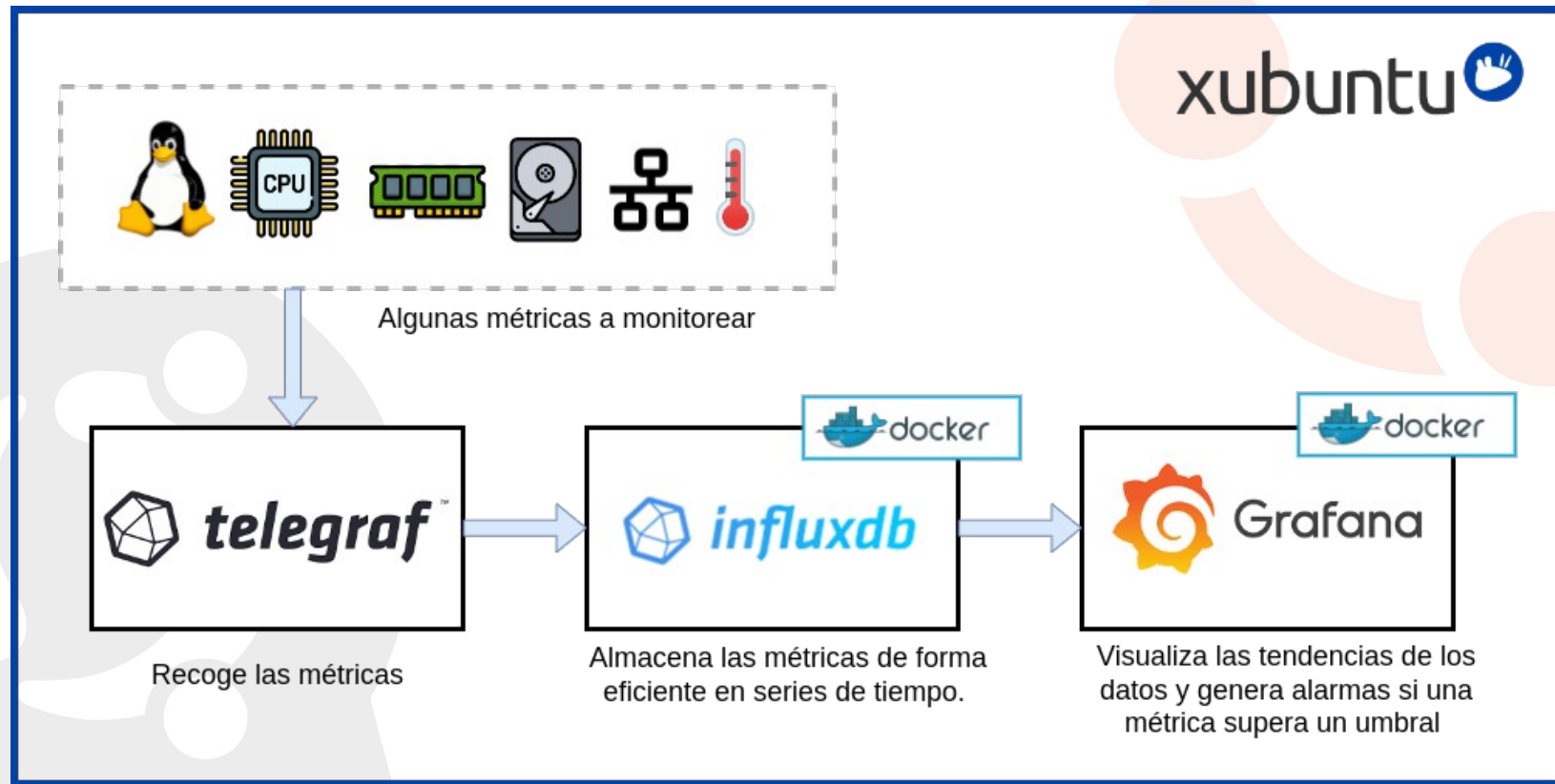
Otras variables que pueden ser monitoreadas

En el desarrollo de APIs:

- Veracidad de los datos.
- Latencia de la API.
- Recursos consumidos por la API.
- Estado y disponibilidad de la API.

Un pipeline de monitoreo con telegraf, influxdb y grafana

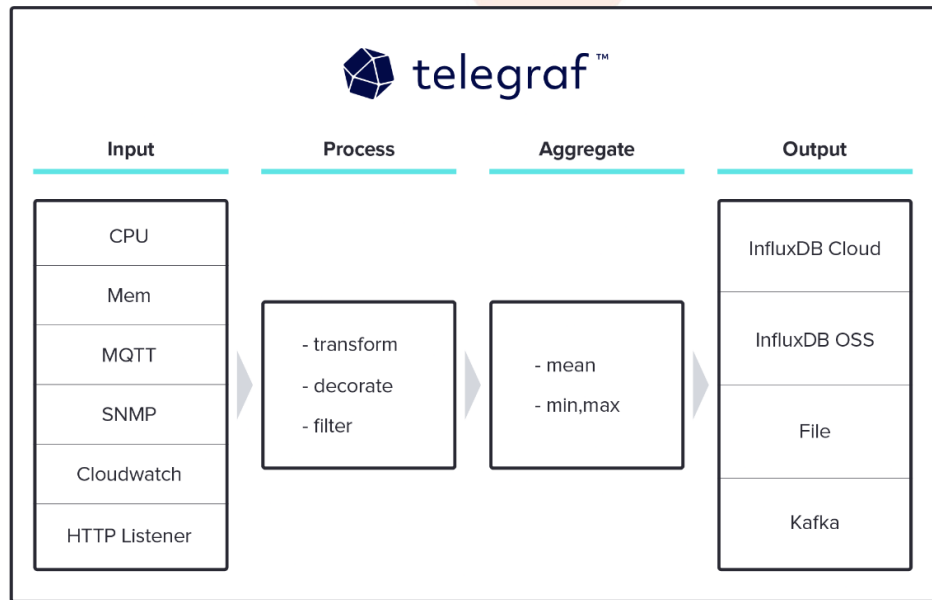
Diseño de la pipeline



Recolectando métricas con **telegraf**

Telegraf es un agente de recopilación de métricas desarrollado por InfluxData.

- Recolecta datos de varias fuentes.
- Fácil de instalar y configurar.
- Ligero y fácil de escalar.



Fuente imagen: [influxdata](https://influxdata.com)

Recolectando métricas con **telegraf**

```
[agent]
  interval = "10s"
  round_interval = true
  metric_batch_size = 1000
  metric_buffer_limit = 10000
  collection_jitter = "0s"
  flush_interval = "10s"
  flush_jitter = "0s"
  precision = ""
  debug = false
  quiet = false
  logfile = ""
  hostname = "${HOSTNAME}"
  omit_hostname = false

[[outputs.influxdb v2]]
  urls = ["http://127.0.0.1:8086"]
  token = "${DOCKER_INFLUXDB_INIT_ADMIN_TOKEN}"
  organization = "${DOCKER_INFLUXDB_INIT_ORG}"
  bucket = "${DOCKER_INFLUXDB_INIT_BUCKET}"
  insecure_skip_verify = true

[[inputs.cpu]]
  percpu = true
  totalcpu = true
  collect_cpu_time = false
  report_active = false
  core_tags = false

[[inputs.linux_cpu]]
```

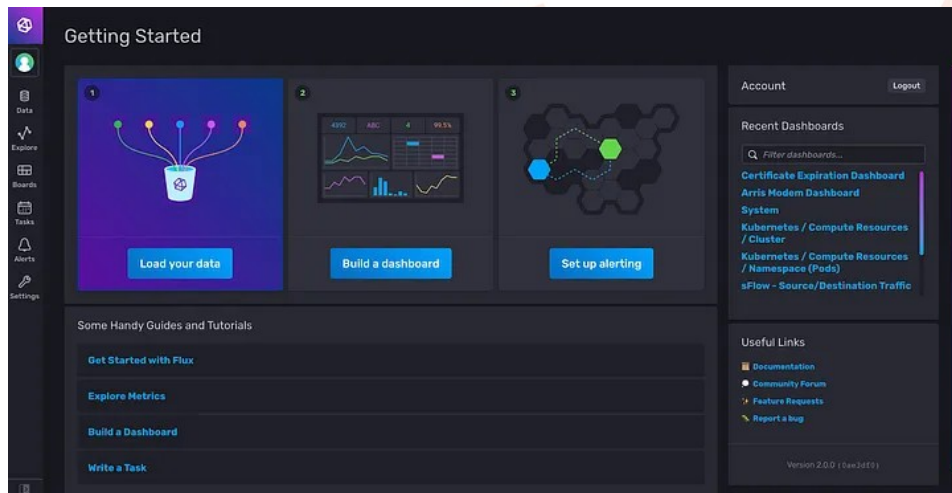
La configuración depende de un archivo de configuración **telegraf.conf** que por defecto debe ser almacenado en el directorio **/etc/telegraf/**

[Aquí](#) un ejemplo de una plantilla de un archivo de configuración.

Almacenamiento de métricas con **influxdb 2**

InfluxDB es una base de datos optimizada para almacenar y consultar series temporales.

- Ideal para manejar y hacer consultas a grandes volúmenes de datos con marca temporal.
- Alta eficiencia de escritura.
- Escalable.



Almacenamiento de métricas con **influxdb 2**

```
influxdb2:
  image: influxdb:2.7.6
  container_name: influxdb2
  ports:
    8086:8086
    8083:8083
  volumes:
    - influxdb2-storage:/var/lib/influxdb
    - influxdb2-config:/etc/influxdb2
  restart: always
  environment:
    - DOCKER_INFLUXDB_INIT_MODE=${DOCKER_INFLUXDB_INIT_MODE}
    - DOCKER_INFLUXDB_INIT_USERNAME=${DOCKER_INFLUXDB_INIT_USERNAME}
    - DOCKER_INFLUXDB_INIT_PASSWORD=${DOCKER_INFLUXDB_INIT_PASSWORD}
    - DOCKER_INFLUXDB_INIT_ORG=${DOCKER_INFLUXDB_INIT_ORG}
    - DOCKER_INFLUXDB_INIT_BUCKET=${DOCKER_INFLUXDB_INIT_BUCKET}
    - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=${DOCKER_INFLUXDB_INIT_ADMIN_TOKEN}
```

La configuración de influxdb es a través de un archivo de configuración. La documentación sobre la configuración se encuentra [aquí](#).

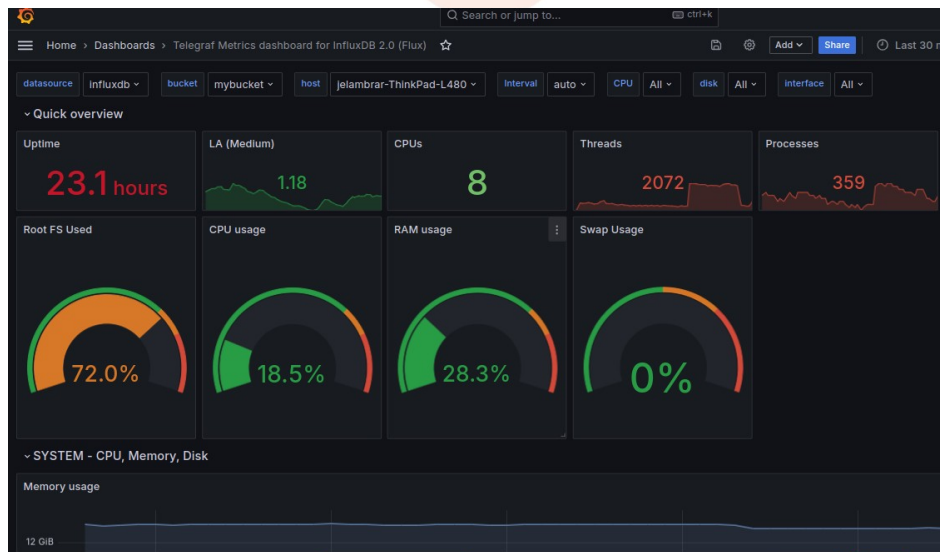
Para el caso de esta pipeline se corre un contenedor **docker** con influxdb 2.0 instalado. Aquí.

<http://localhost:8086/>

Visualizando datos con grafana

Grafana es una plataforma de análisis y monitoreo que permite crear visualizaciones interactivas a partir de diversas fuentes de datos.

- Es ligero y fácilmente configurable
- Diversidad de plugins y fáciles de instalar




```
grafana:
  image: grafana/grafana-oss:main-ubuntu
  container_name: grafana
  ports:
    3000:3000
  volumes:
    grafana-storage:/var/lib/grafana
    ./grafana/datasources:/etc/grafana/provisioning/datasources/
    ./grafana/dashboards:/etc/grafana/provisioning/dashboards/
  restart: unless-stopped
  environment:
    GF_SECURITY_ADMIN_PASSWORD:${GF_SECURITY_ADMIN_PASSWORD}
    - GF_SECURITY_ADMIN_USER:${GF_SECURITY_ADMIN_USER}
```

Visualizando datos con **grafana**

Para el caso de esta pipeline se corre un contenedor **docker** con grafana instalado. Aquí.

<http://localhost:3000/>

Creando una **alerta** en grafana

1. Crear un **webhook** a donde se notificarán las alertas.
2. Crear un nuevo alert rule.
3. Definir la fuente de datos influxdb y el query de la métrica.
4. Definir el nombre de la alerta y el intervalo.
5. En la sección de notificaciones se establece el webhook a donde se enviará la alerta.

Pasos para correr la pipeline

1. Definir las variables de entorno en el archivo **.env**.
2. Instalación y configuración de **telegraf**.
3. Instalación de los servidores de influxdb 2 y grafana usando **docker compose**.
4. Definir como **fuentes de datos** influxdb desde grafana

5. **Importar el tablero** de grafana.
6. Establecer las **alertas** en grafana.

Enlace del repositorio [Github](#)

Enlace de artículo [Medium](#)



echo “Gracias a todos”